

Sistema de Gestão para Escolinhas de Futebol

Cenários de Testes e Relatório

- Bruno Correa Borges Silva
- João Lucas Damato Drummond
- Guilherme Alves Silva





Objetivo do Sistema

O sistema visa otimizar a gestão de escolinhas de futebol, abrangendo funcionalidades de cadastro, financeiro, turmas e comunicação. Nosso foco é aprimorar a eficiência operacional e a experiência de usuários e administradores.

Plano de Testes: Cobertura Abrangente

Nossa estratégia de teste funcional valida as principais funcionalidades do sistema, garantindo robustez e confiabilidade. Utilizamos cenários detalhados para cada módulo.

Cadastro de Aluno

Validação de dados e tratamento de exceções.

Gerenciamento Financeiro

Pagamentos de faturas e criação de planos.

Gestão de Turmas

Adição de alunos e controle de frequência.

Comunicação e Avaliação

Registro de avaliações de desempenho.

1.1. Funcionalidade: Cadastro de Aluno (HU-03)					
ID	Descrição do Cenário	Dado que	Quando	Então	E
1.1	Cadastro com Sucesso	O sistema está inicializado.	O <code>AlunoService.cadastrarAluno</code> é chamado com nome="Novo Aluno" e data válida.	O método retorna um objeto <code>Aluno</code> com ID > 0.	O aluno é salvo no repositório.
1.2	Falha por Nome Vazio	O sistema está inicializado.	O <code>AlunoService.cadastrarAluno</code> é chamado com nome="" (vazio).	O sistema lança uma exceção <code>IllegalArgumentException</code> .	A mensagem é "Nome do aluno não pode ser vazio."
1.3	Falha por Data Futura	O sistema está inicializado.	O <code>AlunoService.cadastrarAluno</code> é chamado com uma data de nascimento futura.	O sistema lança uma exceção <code>IllegalArgumentException</code> .	A mensagem é "Data de nascimento inválida."

1.2. Funcionalidade: Gerenciamento Financeiro (HU-02 & HU-09)					
ID	Descrição do Cenário	Dado que	Quando	Então	E
2.1	Pagamento de Fatura	Existe uma fatura com status PENDENTE.	O <code>FinanceiroService.registrarPagamentoFatura</code> é chamado com a data atual.	O método retorna <code>true</code> .	O status da fatura muda para PAGA e a data de pagamento é registrada.
2.2	Criação de Plano	O repositório de planos está pronto.	O <code>FinanceiroService.criarPlano</code> é chamado com nome="Trimestral", valor=450.00, duração=3.	O método retorna um objeto <code>PlanoPagamento</code> com ID > 0.	O valor e a duração correspondem aos dados informados.

1.3. Funcionalidade: Gestão de Turmas e Frequência (HU-04)					
ID	Descrição do Cenário	Dado que	Quando	Então	E
3.1	Adição de Aluno	Existe uma Turma e um Aluno cadastrados.	O <code>TurmaService.adicionarAlunoNaTurma</code> é chamado com IDs válidos.	O método retorna <code>true</code> .	O aluno é incluído na lista de alunos da turma no repositório.

1.4. Funcionalidade: Comunicação e Avaliação (HU-05 & HU-06)					
ID	Descrição do Cenário	Dado que	Quando	Então	E
4.2	Registro de Avaliação	Um aluno e um treinador existem.	O <code>TurmaService.registrarAvaliacao</code> é chamado com comentários técnicos/táticos.	Um objeto <code>Avaliacao</code> com ID > 0 é retornado.	A avaliação é salva com a data atual.



TDD: Desconto por Pagamento Antecipado – BRUNO

Incentivamos pagamentos em dia com um desconto de 5% para faturas pagas antes do vencimento. Desenvolvido via TDD.

01	02	03
Teste (Red)	Código (Green)	Refatoração
<code>testCalcularValorComDesconto</code> Antecipado falhou.	Lógica if (<code>dataPagamento.isBefore(venci</code> <code>mento))</code> implementada no <code>FinanceiroService</code> .	Extração da constante <code>FATOR_DESCONTO (0.95)</code> .

TDD: Gerar faturas mensais – BRUNO

–Automatiza a criação de faturas para planos ativos, evitando duplicação.



TDD: Limite de Capacidade da Turma - JOÃO

Para manter a qualidade das aulas, limitamos cada turma a 20 alunos.
Esta funcionalidade foi implementada com TDD.

01

Teste (Red)

testAdicionarAlunoTurmaC
heia falhou ao tentar adicionar o
21º aluno.

02

Código (Green)

Verificação if (turma.size() >=
20) adicionada no
TurmaService.

03

Refatoração

Criação da constante CAPACIDADE_MAXIMA.

TDD: Listar fatura - JOÃO

-Exibe apenas faturas pendentes e vencidas.

TDD: Validação de Idade Mínima – GUILHERME

Garantimos que apenas alunos com 4 anos ou mais possam ser cadastrados, assegurando a adequação etária para as atividades.

01

Teste (Red)

`testValidarIdadeMinima`
falhou ao tentar cadastrar uma
criança de 3 anos.

02

Código (Green)

Validação de data de nascimento
adicionada no `AlunoService`.

03

Refatoração

Criação da constante `IDADE_MINIMA`.

TDD: Calcular Total Pago – GUILHERME

Soma apenas faturas com status PAGA.

